

RetailLake

An AWS-based retail data engineering pipeline built using Amazon S3, AWS Glue, Glue Data Catalog and Amazon Athena.

RetailLake processes raw retail datasets through a Raw → Silver → Gold architecture. The project focuses on data quality, incremental file ingestion, transformation, orchestration and analytical data preparation.

1. Project Overview

I had previously worked with tools such as Airflow and Docker, where I experienced data engineering using multiple components and locally managed infrastructure.

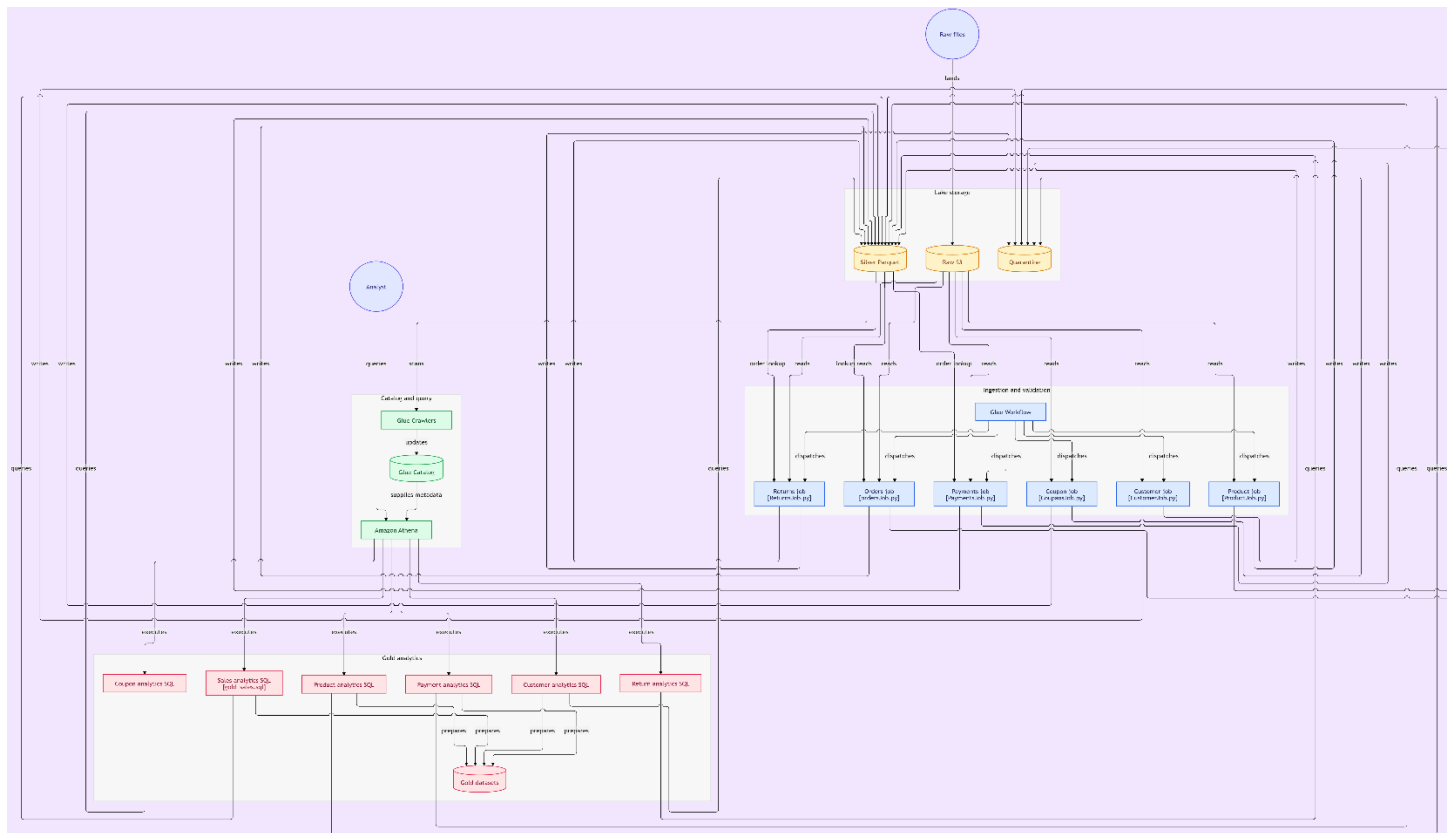
With RetailLake, I wanted to explore how an end-to-end data pipeline could be built using AWS-managed services.

The project stores incoming data in Amazon S3, processes it using AWS Glue and PySpark, stores cleaned data in Parquet format, maintains metadata through Glue Crawlers and Glue Data Catalog, and provides analytical datasets that can be queried using Athena.

The project also focuses on practical data engineering problems such as:

- Data validation
- Duplicate detection
- Referential integrity
- Data quarantine
- Parquet storage
- Partitioning
- Incremental file ingestion
- Job orchestration
- Schema management
- Analytical data preparation

2. Architecture



Service	Purpose
Amazon S3	Data lake storage
AWS Glue	PySpark-based ETL and orchestration
AWS Glue Data Catalog	Metadata and table definitions
AWS Glue Crawlers	Discover and update Silver dataset metadata
AWS Glue Workflows	Control job dependencies
AWS Glue Job Bookmarks	Incremental file ingestion
Amazon Athena	SQL querying and analytical validation
IAM	Access control and permissions

4. Data Lake Layers

Raw

The Raw layer contains the incoming source data in its original form.

s3://retaillake-data/raw/

The Raw layer is intentionally not treated as the cleaned analytical layer.

Silver

The Silver layer contains validated and transformed data stored as Parquet.

s3://retaillake-data/silver/

Silver processing includes:

- Null handling
- Duplicate detection
- Primary/business key validation
- Date validation
- Regex/pattern validation
- Referential integrity
- Business-rule validation
- Data type conversion
- Derived columns
- Dataset enrichment

Only records that pass the required validation rules are written to the valid Silver datasets.

Quarantine

Invalid records are not immediately deleted.

Instead, they are separated into:

s3://retaillake-data/quarantine/

Each invalid record contains information such as:

- record_status
- validation_reason

This makes it possible to understand why a record failed instead of simply losing the source data.

The quarantine data can also be used to identify recurring data-quality problems and investigate whether the issue originates in the source data or the transformation logic.

Gold

The Gold layer contains business-oriented analytical datasets.

s3://retaillake-data/gold/

The Gold layer is designed around commonly required analytical questions so that the same joins and aggregations do not need to be recreated every time.

5. Source Datasets

RetailLake processes six datasets:

- Customers
- Products
- Coupons
- Orders
- Payments
- Returns

The datasets have relationships between them.

Customers, Products and Coupons can exist independently.

Orders depend on relevant customer and product records. Coupons are optional because only some orders use coupons.

Payments and Returns depend on order_id.

6. Silver Data Quality

The Silver jobs apply several categories of validation.

Null validation

Required fields are checked for missing values.

For example, customer records require a `customer_id`, while important attributes such as email and phone are validated according to the dataset rules.

Duplicate validation

The pipeline checks for:

- Duplicate complete rows
- Duplicate primary/business keys

Date validation

Date fields are converted to appropriate date types rather than being left as strings.

Dates are also checked against business rules, such as preventing registration or transaction dates from being in the future when such dates are invalid.

Pattern validation

Regex-based validation is used for fields such as:

- Phone numbers
- Email addresses
- Other structured identifiers where applicable

Referential integrity

Relationships between datasets are validated before a record is considered valid.

For example:

`Orders.customer_id` → `Customers.customer_id`

`Orders.product_id` → `Products.product_id`

`Payments.order_id` → `Orders.order_id`

`Returns.order_id` → `Orders.order_id`

This prevents records with missing parent references from entering the valid Silver layer.

Business rules

Dataset-specific rules are also applied to fields such as amounts, dates and other business conditions.

7. Derived Silver Columns

The Silver layer also creates fields that represent commonly required business concepts.

Examples include:

Customers

- customer_age_days
- customer_type
- year
- month
- day

Products

- product_tier
- price_band
- category

Coupons

- discount_category
- status
- days_until_expiry
- expiry_year
- expiry_month

Orders

- amount_band
- order_age_days
- order_year
- order_month
- quarter

Orders are also enriched with relevant customer, product and coupon information.

Payments

- payment_category

- payment_age_days
- year
- month

Returns

- return_age_days
- year
- month

These fields reduce the need to repeatedly perform the same calculations during analysis.

8. Parquet Storage

The Silver layer uses Parquet instead of continuing to store transformed data as CSV.

Parquet is a columnar format, which is useful for analytical workloads.

For example, if a query only needs to filter or aggregate using a column such as amount, a columnar format can avoid reading unnecessary columns.

This reduces the amount of data that needs to be scanned compared with a row-oriented representation such as CSV and also provides more efficient storage for analytical workloads.

9. Partitioning

Partitioning was designed according to the characteristics and expected use of each dataset rather than applying the same partition strategy everywhere.

Dataset	Partitioning
Customers	Year / Month / Day
Coupons	Expiry Year / Expiry Month
Orders	Order Year / Order Month
Payments	Year / Month
Returns	Year / Month

Dataset	Partitioning
Products	Category

Time-based partitioning is useful for datasets where dates are important to analysis.

Products are instead partitioned by category because date-based partitioning does not provide the same benefit for this relatively static dataset.

10. Incremental Ingestion with Glue Job Bookmarks

Initially, the Glue jobs were not using Job Bookmarks.

Bookmarks were then enabled and the bookmark state was reset before testing to establish a clean starting point.

The behavior was tested by:

1. Running the job against the existing Raw data.
2. Adding new files to the S3 input path.
3. Running the job again.
4. Checking that only the newly introduced input was processed.
5. Resetting the bookmark and repeating the tests where required to verify that previously processed data was not unnecessarily duplicated.

The bookmark testing was performed across the six ingestion jobs.

What Job Bookmarks provide

For the S3 ingestion used in RetailLake, Job Bookmarks maintain state about previously processed input so that subsequent runs can process newly arrived files incrementally.

This means the pipeline does not need to repeatedly process the same previously processed files.

Important limitation

Job Bookmarks provide incremental **file-level ingestion**. They do not automatically provide row-level change detection or upsert behavior.

The new file can be processed, but the bookmark does not automatically determine that the existing Silver record for customer should be updated.

Handling this type of cross-batch update is a separate requirement.

11. Glue DynamicFrames

An important implementation detail was the use of Glue DynamicFrames for S3 ingestion where bookmark behavior was required.

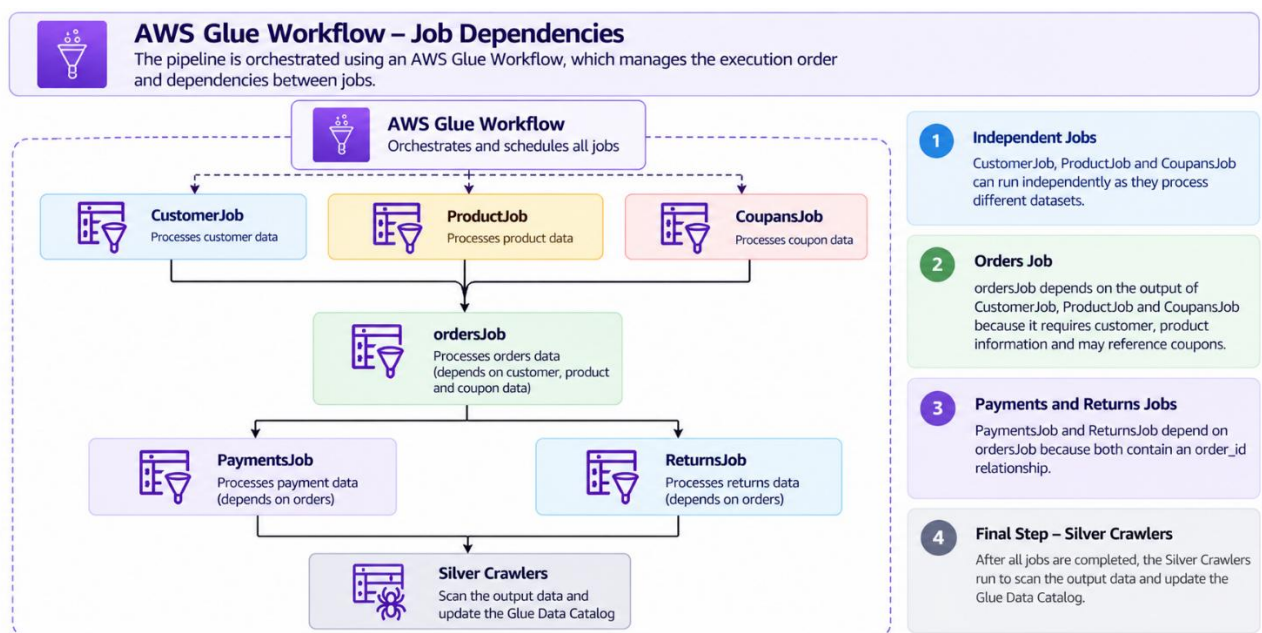
The initial implementation used Spark's DataFrame reader directly, but the expected bookmark behavior was not obtained.

The ingestion was changed to use Glue DynamicFrames with a `transformation_ctx`, allowing the Glue job to maintain bookmark state for the source.

12. AWS Glue Workflow

The pipeline is orchestrated using an AWS Glue Workflow.

The dependency structure is:



Customer, Product and Coupon processing can occur independently.

Orders depends on these datasets because it requires customer/product information and may reference coupons.

Payments and Returns depend on Orders because both contain an `order_id` relationship.

The Silver crawlers run after the relevant transformations have completed.

The workflow can be started manually when required, while a scheduled start trigger can be activated when scheduled execution is desired.

13. Glue Crawlers and Data Catalog

Glue writes the transformed Parquet files to S3, but Athena requires metadata describing those datasets.

Glue Crawlers examine the Silver data and maintain the Glue Data Catalog so that Athena knows information such as:

- Columns
- Data types
- Dataset location
- Table metadata

The crawlers therefore form the metadata layer between the Silver S3 data and Athena.

Schema evolution is a consideration for future versions of the project. The current implementation did not encounter an incorrect crawler schema inference, but changes in source structure should be monitored as the pipeline evolves.

14. Gold Analytics

RetailLake contains six analytical Gold datasets.

gold_sales

Provides enriched order-level sales information including customer, product and coupon information.

gold_customer_analytics

Used to analyze customer ordering and purchasing behavior.

gold_product_analytics

Used to understand how products are performing, including sales, revenue and return-related metrics.

gold_payment_analytics

Used to analyze payment methods, payment status and the relationship between payment and order amounts.

gold_return_analytics

Used to analyze return frequency, return reasons/categories and customer-related patterns.

gold_coupon_analytics

Used to analyze coupon usage, orders associated with coupons and sales generated through coupon usage.

The Gold layer reduces the need to repeatedly perform the same joins and aggregations directly against the Silver layer.

15. Validation of the Final Pipeline

The final Silver datasets were validated for:

- Row counts
- Null values
- Referential integrity
- Duplicate behavior
- Schema correctness
- Quarantine output

The final Gold datasets were validated for:

- Data availability
- Key uniqueness
- Basic metric correctness
- Enrichment
- Sample records
- Reconciliation against Silver where applicable

Example final Silver row counts:

Dataset	Silver records
Customers	1,897
Products	935
Coupons	187
Orders	5,746
Payments	4,612
Returns	1,162

16. Problems Encountered and Solutions

Building RetailLake involved several debugging problems across AWS services.

16.1 Job Bookmarks were not working with the initial ingestion approach

The initial implementation used Spark's DataFrame reader directly.

After testing, the expected incremental bookmark behavior was not obtained.

The ingestion was changed to Glue DynamicFrames with a `transformation_ctx`, after which bookmark behavior was tested by adding new files and verifying incremental processing.

16.2 JSON files containing an outer array

Some JSON files were structured as:

```
[
  {
    "coupon_id": 300,
    "coupon_code": "SAVE927NR",
    "discount_percent": 20,
    "expiry_date": "2026-07-22"
  }
]
```

The initial DynamicFrame configuration did not correctly interpret the objects inside the outer array as individual records.

The solution was:

```
format="json",  
format_options={  
    "multiline": True,  
    "jsonPath": "$[*]"  
}
```

This allowed each object in the outer array to be processed as an individual record.

16.3 Numeric schema inference

Some JSON numeric fields were inferred as a structure containing multiple possible numeric representations:

```
struct<double: DOUBLE, int: INT>
```

rather than directly as:

```
double
```

The schema was inspected and the transformation handled the possible representations before converting the final value to the expected numeric type.

16.4 Gold table contained stale data

During reconciliation, the Silver Orders dataset contained 5,746 records while the physical Gold Sales table initially contained 5,742.

The Gold SQL query itself returned 5,746 records, showing that the query logic was not the source of the discrepancy.

The issue was that the physical Gold table was based on an earlier result.

A corrected Gold dataset was generated separately, verified, and the table was updated to use the corrected data.

17. Cross-Batch Duplicate and Upsert Limitation

The current pipeline provides incremental file ingestion but does not implement full cross-batch upsert semantics.

For example, if a later file contains a changed version of an existing business key, the current pipeline can process the new file but does not automatically determine whether the previous Silver record should be updated or replaced.

A production implementation for high-volume datasets requiring frequent updates could use a lakehouse format such as Apache Iceberg and MERGE-based processing.

This was intentionally kept as a future enhancement rather than adding unnecessary complexity to the current project.

18. Lessons Learned

RetailLake helped reinforce several practical data engineering concepts.

Design before implementation

Establishing naming conventions, input/output paths, partitioning strategies and schema expectations earlier would make the implementation cleaner.

Use the appropriate Glue ingestion mechanism

When Glue-specific features such as Job Bookmarks are required, the source implementation needs to use the corresponding Glue mechanisms rather than assuming standard Spark readers will provide the same behavior.

Data quality needs observability

Tracking:

- Total records
- Valid records
- Invalid records
- Validation/success rate

would make batch-level monitoring more useful.

Debugging is not limited to code

Some of the most difficult problems required checking multiple components together:

S3

↓

Glue Job



Bookmark State



Parquet Output



Crawler



Glue Catalog



Athena

A problem appearing in Athena, for example, could originate from the physical S3 data or an earlier transformation rather than the SQL query itself.

19. Future Improvements

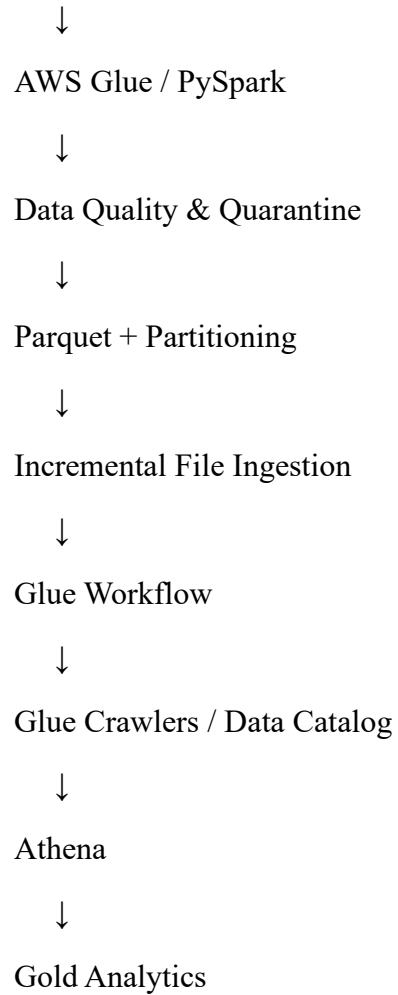
Possible future improvements include:

- Better configuration management instead of hardcoded S3 paths
- More consistent naming conventions
- S3 Versioning enabled from the beginning
- More automated batch-level testing and quality metrics
- Better schema-evolution handling
- Cross-batch upserts using Apache Iceberg
- More comprehensive monitoring and alerting

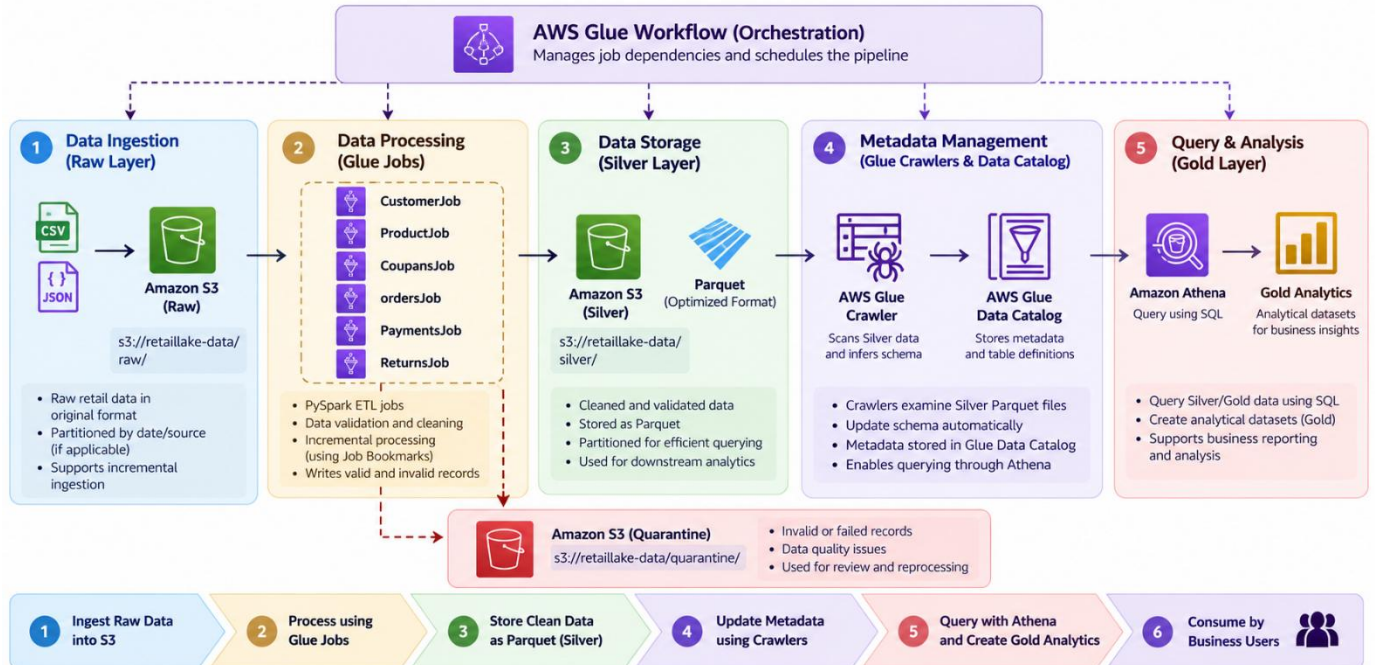
20. Project Outcome

RetailLake demonstrates an end-to-end AWS data engineering workflow covering:

S3 Storage



The project was built to understand not only how to use individual AWS services, but also how they interact as part of a complete data pipeline.



AWS Services

- Amazon S3
- AWS Glue
- AWS Glue Data Catalog
- AWS Glue Crawlers
- AWS Glue Workflows
- AWS Glue Job Bookmarks
- AWS IAM
- Amazon Athena